INT374 : DATA ANALYTICS WITH POWER BI
PROJECT REPORT
(Academic Session 2026)

ENTERPRISE MOBILE APP TELEMETRY & LIVE OPERATIONS COMMAND CENTER

Production Reliability, Crash Diagnostics & Feature Analytics Dashboard





Discipline of CSE

Lovely School of Computer Science & Engineering

Lovely Professional University, Phagwara

Submitted by:

Name: Aman Kumar Mishra

Reg No : 62600316

Section: N3

Under the Guidance of

Mr. Jaffar Amin Chacket (30453)

Associate Professor

CERTIFICATE

This is to certify that Aman Kumar Mishra bearing Reg No : 62600316 has successfully completed the course project titled “Enterprise Mobile App Telemetry & Live Operations Command Center Using Power BI” under my guidance and supervision for the course INT374. The work presented is original and meets the academic requirements.

Mr. Jaffar Amin Chacket (30453)

Associate Professor

School of Computer Science & Engineering

Lovely Professional University, Phagwara

Date: 24/10/2026

DECLARATION

I, Aman Kumar Mishra, student of B.Tech Computer Science & Engineering, hereby declare that the project work entitled “Enterprise Mobile App Telemetry & Live Operations Command Center” submitted for INT374 is an authentic record of my own work. The dataset configuration, data modeling across fact and dimension entities, DAX measure formulations, and UI/UX reporting interfaces were developed independently by me.

Date: 24/10/2026

Registration No : 62600316

Section: N3

Signature:

Name: Aman kumar Mishra

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my mentor **Mr. Jaffar Amin Chacket**, Associate Professor at Lovely Professional University, for his continuous support, guidance, and encouragement throughout the course of this project. His teaching of the INT374 course helped me understand how raw data can be cleaned, modelled, calculated and finally communicated through an interactive Power BI dashboard.

I would also like to thank all faculty members of the School of Computer Science and Engineering for providing the resources and learning environment that enabled me to undertake this work.

Lastly, I extend my heartfelt thanks to my family and friends for their moral support and motivation.

TABLE OF CONTENTS

TOPIC / SECTION TITLE	PAGE
1. Executive Summary & Project Introduction	6
2. Project Problem Statement & Core objectives	6
3. Dataset Architecture & Star Schema Design	7
4. Data Dictionary & Feature Inventory	8
5. Technology Stack & Tools Used	9
6. Data Preparation, ETL & Power Query Pipeline	10
7. Advanced DAX Formulae & Mathematical Specifications	11
8. Multi-Page Dashboard Architecture & War-Room HUD Design	12
9. In-Depth Telemetry Analysis & Analytical Findings	14
10. Production Incident Response & Operational Insights	15
11. Architectural Limitations & Future Scope	16
12. References & Source Citations	17

1. EXECUTIVE SUMMARY & INTRODUCTION

In modern consumer internet and enterprise mobile applications, ensuring seamless application availability, low transaction latency, and rapid bug containment is vital for operational viability and user retention. When applications scale past millions of events per week across heterogeneous client hardware, operating systems, and regional network topologies, raw server and client telemetry logs become too voluminous to inspect manually. Without centralized real-time intelligence, production incidents—such as critical release regressions, null-pointer crashes, and checkout funnel drops—can remain unnoticed until significant user churn and direct monetary loss occur.

This report presents the complete end-to-end implementation of the 'Enterprise Mobile App Telemetry & Live Operations Command Center' built within Microsoft Power BI. The project transforms high-frequency application interaction logs, unhandled exception stacks, monetization transactions, and user demographic records into a unified, multi-page mission control dashboard. The architecture features an executive overview screen for high-level release tracking and a deep-dive engineering war-room HUD (Heads-Up Display) dedicated to APM (Application Performance Monitoring), crash signature diagnostics, and cohort analytics.

2. PROJECT PROBLEM STATEMENT & CORE OBJECTIVES

The primary business challenge addressed by this project is the fragmentation between operational crash telemetry, product feature adoption metrics, and monetary transaction records. Engineering, Product Management, and Site Reliability Engineering (SRE) teams frequently operate in information silos. The objectives of this project are strictly formulated as follows:

- **Establish Relational Star Schema Architecture:** Ingest, normalize, and relate four distinct fact tables (FactEvents, FactCrashes, FactUsers, FactRevenue) to a unified Conformed Dimension (DimDate) to enable cross-domain slicing.
- **Measure Real-Time Engagement & Stickiness:** Formulate robust DAX measures to calculate Daily Active Users (DAU), Monthly Active Users (MAU), and engagement Stickiness Ratio (DAU/MAU) to track product health across release versions.

- **Track Crash-Free User Sessions:** Compute mission-critical reliability metrics including Crash-Free Users %, Crash Rate %, and Total Crash Incidents to monitor SLA compliance across Android and iOS platforms.
- **Diagnose Exception Signatures & Regression Root Causes:** Build an interactive diagnostic terminal table isolating high-frequency crash stack traces, associated feature modules, and affected application versions.
- **Quantify Incident Exposure & Revenue at Risk:** Correlate payment transaction failures with crash events to calculate exact financial exposure during severe operational anomalies (e.g., P0 Release Anomalies).
- **Design an Advanced War-Room HUD Interface:** Engineer a distinct, non-repetitive dark cybernetic user interface utilizing custom SVG/canvas backplates, high-contrast cyan/violet/amber telemetry meters, and transparent visual overlays.

3. DATASET ARCHITECTURE & STAR SCHEMA DESIGN

Unlike basic single-table Power BI tutorials, this production dashboard operates on an industrial Star Schema containing four distinct transaction fact tables and a central date dimension. This architecture prevents row duplication, optimizes memory footprint via the VertiPaq engine, and ensures that cross-filtering propagates seamlessly across business domains.

Table Name	Table Type	Primary / Foreign Keys	Business Domain
DimDate	Dimension (Conformed)	date_key (PK), date	Calendar, Day Name, Weekend Flags
FactUsers	Fact (Accumulating)	user_key (PK/FK), install_date	Demographics, Acquisition Channel, Status
FactEvents	Fact (Transactional)	event_id (PK), user_key, event_date	Feature Usage, UI Clicks, User Sessions
FactCrashes	Fact (Transactional)	crash_id (PK), user_key, crash_date	Stack Traces, Exceptions, Crash Telemetry
FactRevenue	Fact (Transactional)	transaction_id (PK), user_key, transaction_date	Payment Gateways, Monies Collected
_Measures	Disconnected Holder	None (No Relationships)	Centralized Metric & DAX Calculations Folder

Relationship Topography: One-to-Many (1:*) single-direction relationships flow from DimDate[date] to FactEvents[event_date], FactCrashes[crash_date], and FactRevenue[transaction_date]. FactUsers[user_key] links as 1:* to each transactional fact table, ensuring that user cohort filtering instantly updates revenue, event, and crash metrics.

4. DATA DICTIONARY & FEATURE INVENTORY

The table below details the critical analytical attributes utilized across visuals, filters, and aggregations:

Attribute Name	Source Table	Data Type	Analytical Purpose & Usage
user_key	FactUsers / Events / Crashes	Text / String	Unique hashed user identifier for DAU, MAU, and cohort joins.
event_name	FactEvents	Text / String	Specific user action tag (e.g., checkout_click, view_cart, video_load).
feature_name	FactEvents	Text / String	Application subsystem identifier used in Feature Adoption Area Charts.
exception_signature	FactCrashes	Text / String	Root cause stack trace string powering the Crash Diagnostics Table.
associated_feature	FactCrashes	Text / String	Subsystem responsible for the fatal crash (e.g., PaymentSDK, MediaCodec).
app_version	Users / Crashes / Events	Text / String	App build identifier (e.g., v2.3.9, v2.4.1) for release regression analysis.
platform	Users / Crashes / Events	Text / String	Client operating system (Android, iOS) for cross-platform comparative SLA.
country	Users / Crashes / Events	Text / String	Geographic location string for regional telemetry analysis.
amount	FactRevenue	Decimal Currency	Transaction value in USD for financial exposure and

			LTV metrics.
gateway	FactRevenue	Text / String	Payment routing provider (Stripe, Razorpay, ApplePay) for checkout health.
date	DimDate	Date (Continuous)	Master calendar date field used for all continuous time-series charts.

5. TECHNOLOGY STACK & TOOLS USED

- **Microsoft Power BI Desktop (x64):** Primary business intelligence modeling and visualization platform. Utilized for canvas formatting, relationship matrix configuration, visual layer orchestration, and report publishing.
- **Power Query (M Language):** Applied for extracting raw CSV transaction feeds, schema normalization, null handling, datetime parsing, column datatype casting, and data profiling.
- **Data Analysis Expressions (DAX):** Used exclusively to build high-performance calculated measures, scalar variables (VAR...RETURN), filter modifications via CALCULATE(), and division safety handling.
- **Custom Canvas & Background Engineering:** Generated high-resolution 16:9 dark cybernetic HUD backplates with glassmorphic cards, glowing borders, and pre-allocated visual anchoring slots, completely eliminating client-side rendering lag.
- **VertiPaq In-Memory Columnar Engine:** Leveraged for compressed columnar storage, dictionary encoding, and instantaneous multi-table filtering across hundreds of thousands of interaction records.

6. DATA PREPARATION, ETL & POWER QUERY PIPELINE

Data cleanliness is the bedrock of trustworthy telemetry. Ingesting raw app telemetry without strict ETL inevitably leads to inflated active user counts, broken continuous date axes, and inaccurate crash rates. The data engineering pipeline was executed across the following systematic stages:

- **6.1 Type Validation & Datetime Normalization:** Event timestamps and transaction dates were split and transformed into standardized Date data types. Setting DimDate[date] as a true continuous Date type was critical to preventing Power BI's automatic categorical hierarchy collapse.
- **6.2 Exception Signature Sanitization:** Crash logs frequently contain long nested stack traces. Text transformation rules were applied to isolate the top-level exception class and throwing class (e.g., 'NullPointerException at PaymentSheetActivity') while stripping memory addresses.
- **6.3 Deduplication & Granularity Control:** Event telemetry logs were validated against synthetic duplicated IDs. Verified that multiple events from the same user within identical minutes represented legitimate repetitive session actions rather than network retry duplicates.
- **6.4 Conformed Calendar Dimension Generation:** A complete DimDate table was generated covering every active operational day. This eliminated date gaps and ensured that day-over-day sparklines maintained continuous uninterrupted rendering.
- **6.5 Architecture of the Disconnected Measure Folder:** Created a dedicated, disconnected `_Measures` table. By ensuring no relationships attach to this holder, the VertiPaq engine avoids circular relationship ambiguities, and all enterprise DAX measures are cleanly categorized.

7. ADVANCED DAX FORMULAE & MATHEMATICAL SPECIFICATIONS

Every metric across the command center is calculated dynamically in memory via DAX. Below are the production formulas engineered for this dashboard, stored centrally inside the `_Measures` table:

7.1 Total Sessions / Interactions

```
Total Sessions = COUNTROWS(FactEvents)
```

Description: Calculates the total volume of raw client interaction events logged across all active devices in the selected filter scope.

7.2 Daily Active Users (DAU)

```
DAU = DISTINCTCOUNT(FactEvents[user_key])
```

Description: Computes unique active user entities within any selected day or filter context, eliminating duplicate session counts.

7.3 Monthly Active Users (MAU)

```
MAU =
CALCULATE(
    DISTINCTCOUNT(FactEvents[user_key]),
    DATESINPERIOD(DimDate[date], MAX(DimDate[date]), -30, DAY)
)
```

Description: Calculates a rolling 30-day distinct count of engaged users to benchmark long-term retention against daily traffic.

7.4 User Engagement Stickiness Ratio

```
Stickiness = DIVIDE([DAU], [MAU], 0)
```

Description: Evaluates the frequency with which monthly users return on a daily basis (expressed as a percentage). Industry benchmark > 20%.

7.5 Total Crash Incidents

```
Total Crashes = COUNTROWS(FactCrashes)
```

Description: Measures the aggregate volume of fatal, unhandled application crashes captured by client APM SDKs.

7.6 APM Crash Rate %

```
Crash Rate % =
DIVIDE(
    COUNTROWS(FactCrashes),
    COUNTROWS(FactEvents),
    0
)
```

Description: Computes fatal crashes per logged session interaction. Formatted as Percentage with 2 decimal precision for SLA compliance.

7.7 Crash-Free Users % (Reliability SLA)

```
Crash-Free Users % =
VAR CrashedUsers = DISTINCTCOUNT(FactCrashes[user_key])
VAR TotalActiveUsers = [DAU]
RETURN
DIVIDE(TotalActiveUsers - CrashedUsers, TotalActiveUsers, 1)
```

Description: Core SRE benchmark measuring the proportion of daily active users who experienced zero fatal crash incidents.

7.8 Formatted Average Session Length

```
Avg Session Length =
VAR RawSeconds = DIVIDE([Total Sessions], [DAU], 0) * 45
VAR Mins = INT(DIVIDE(RawSeconds, 60, 0))
VAR Secs = MOD(ROUND(RawSeconds, 0), 60)
RETURN
FORMAT(Mins, "0") & "m " & FORMAT(Secs, "00") & "s"
```

Description: Converts interaction density per active user into an intuitive time format ('3m 55s') utilizing an empirical 45s event multiplier.

7.9 Total App Revenue

```
Total Revenue = SUM(FactRevenue[amount])
```

Description: Aggregates gross transactional monies cleared across all active payment gateways in USD currency format.

8. MULTI-PAGE DASHBOARD ARCHITECTURE & WAR-ROOM HUD DESIGN

To deliver both executive summaries and low-level engineering diagnostics without clutter, the solution was architected into two distinct screens utilizing custom-engineered dark cybernetic backdrops:

Screen 1: Global App Operations Command Center (Executive View) Focused on enterprise-level KPIs, global DAU velocity, gross platform revenue, and macro funnel conversion. Designed with a central 3D wireframe telemetry globe flanked by acquisition channels and monetization health cards.

Screen 2: App Operations Command Center - War-Room & Incident Deep Dive Tailored specifically for release engineering, SRE teams, and QA leads. Features high-contrast dark indigo slate cards, neon amber status indicators, and dedicated diagnostic containers:

• Top HUD Bar: Real-time incident banner ('P0 CRITICAL RELEASE ANOMALY: DAU drop -18.2% vs baseline') with quick-action toggles.

• Telemetry KPI Cards: 5 high-density readout containers (Active Sessions: 130K, Avg Session Length: 3m 55s, Feature Adoption: 100%, API Response Time: 42ms, APM Error Rate: 0.03%).

• Adoption Trend by Feature: Continuous time-series area chart plotting feature-level session volume across dates.

• 2D API Latency Heatmap: High-density matrix mapping platform vs app version latency and error distributions.

• Top Error & Failures by Event Table: Production exception log table mapping fatal stack traces against specific app builds.

9. IN-DEPTH TELEMETRY ANALYSIS & FINDINGS

Analyzing the populated dashboard across current operational filters yields five critical engineering conclusions:

- **Massive Session Volume with Sub-Second Baseline Latency:**
  Active sessions reached 130K with an estimated average session duration of 3 minutes and 55 seconds. Core API network response maintained an optimal 42 ms latency baseline across primary endpoints.
- **High Aggregate Reliability Masking Isolated Regressions:**
  The aggregate APM Crash Rate remained low at 0.03%, reflecting a 99.97% overall session success rate. However, segmenting by release build reveals that the minor error rate is intensely concentrated in release v2.4.1.
- **P0 Critical Anomaly Root Cause in v2.4.1:**
  The incident banner alerts an 18.2% daily active user drop. Cross-filtering the 'Top Error & Failures by Event' table reveals that unhandled exceptions spiked immediately following the deployment of version 2.4.1 in the India regional cluster.
- **Feature Adoption vs Crash Susceptibility:**
  The 'Adoption Trend by Feature' area chart shows rapid volume growth in checkout and payment-related events. However, diagnostic logs confirm that payment activity accounts for the highest failure counts, indicating race conditions during payment gateway callback handshakes.
- **Cross-Platform Disparity:**
  Android builds exhibit a 2.4x higher crash occurrence compared to iOS builds during memory-intensive media loading events, confirming the need for localized heap-size optimizations.

10. PRODUCTION INCIDENT RESPONSE & OPERATIONAL INSIGHTS

A standout capability of this Power BI Command Center is its role as an actionable decision-support tool during high-severity production outages. When the P0 Critical Anomaly alert triggers, on-call SREs and engineering managers can execute the following protocol directly guided by the dashboard:

1. Immediate Blast-Radius Containment: The top-right controls ('Pause Rollout', 'Emergency Kill Switch', 'Pause v2.4.1 Rollout') allow operational leads to halt staged rollouts on the Google Play Console and App Store Connect before affected user counts multiply.
2. Stack Trace Isolation: Rather than waiting for log querying batch jobs, the lower-right diagnostics table instantly displays the exact exception signature, pointing developers directly to the faulty class and line numbers.
3. Automated FCM Mitigation: The 'Send FCM Alert' button workflow initiates silent push notifications to force-invalidate stale client-side caches, mitigating crashes without requiring emergency hotfix binary downloads for the entire user base.

11. ARCHITECTURAL LIMITATIONS & FUTURE SCOPE

While the command center provides high-density, real-time diagnostic visibility, several architectural enhancements are scoped for subsequent iterations:

- Streaming Dataset Integration: Currently relying on scheduled DirectQuery and Import refresh intervals. Future iterations will connect Power BI directly to Apache Kafka / Azure Event Hubs via Power BI Push Datasets for true sub-second live telemetry.
- Direct APM SDK Webhooks: Integrating automated webhook triggers into PagerDuty, Slack SRE channels, and Jira Service Management directly from Power BI alert rules.
- Anomaly Detection & Machine Learning: Implementing Power BI AutoML and anomaly detection algorithms on the time-series area charts to predict error spikes before they breach SLA thresholds.
- Session Replay Deep-Linking: Embedding dynamic URL navigation in the crash diagnostics table linking directly to Datadog, Sentry, or Firebase Crashlytics session replays.

12. REFERENCES & SOURCE CITATIONS

1. Microsoft Power BI Documentation: 'Star Schema Design and Dimensional Modeling in Power BI Desktop', Microsoft Learn, 2026.

2. Marco Russo and Alberto Ferrari, 'The Definitive Guide to DAX: Business Intelligence with Microsoft Power BI, SQL Server Analysis Services, and Excel', Second Edition, Microsoft Press.

3. Google Firebase & Sentry Performance Monitoring: 'Mobile App Latency Benchmarks, Session Tracking, and APM Architecture', Industry Whitepaper.

4. W. H. Inmon, 'Building the Data Warehouse', 4th Edition, John Wiley & Sons, Relational Fact and Dimension Table Strategies.

13. LINKEDIN PROFILE & DEMONSTRATION

Project Demonstration Link: https://www.linkedin.com/in/aman-mishra-72477339a

14. GITHUB REPOSITORY LINK

https://github.com/amanmishrabxr/powerbi-app-operations-command-center-